Java concurrency based on the Devoxx Poland 2024 session by Jarosław Pałka and Andrzej Grzesik.

TL;DR: The Concurrency Hierarchy

Locking in the JVM is not a single mechanism but a spectrum of performance vs. complexity. **Synchronized** is handled by the JVM (slow, "fat" locks), **ReentrantLock** is handled by Java calling the OS (moderate), and **Atomics** are handled directly by the CPU (fastest). As concurrency increases, traditional `synchronized` blocks suffer from "stranding" (where threads stay parked even if the lock is free), leading to throughput degradation.

---

1. The Anatomy of `synchronized`: "Lies to Children"

The presenters describe the common explanation of `synchronized` as "lies to children"—a simplified model that masks extreme internal complexity.

- **Object Header Layout:** Every Java object has a "Mark Word" in its header. This word stores state bits that define the locking status:
- **01:** Unlocked (also stores hashcode and age).
- **00:** Lightweight locked (pointer to stack record).
- **10:** Heavyweight locked (pointer to an `ObjectMonitor`).
- **Lock Inflation Path:**

1. **Stack/Lightweight Locking:** The JVM first tries to record the lock on the thread's stack.
2. **Inflation:** If a second thread tries to acquire the lock, it "inflates" to a **Fat Lock**.
3. **ObjectMonitor:** A C++ structure that manages a queue of waiting threads.

- **The "Stranding" Problem:** In highly contended `synchronized` blocks, the "responsible thread" (the one tasked with waking others) may face delays (up to 10-1000ms), leaving the lock empty while threads remain "stranded" in a parked state.

2. Lock Support & AbstractQueuedSynchronizer (AQS)

When you move away from `synchronized` to `java.util.concurrent.locks`, you move the logic from the JVM (C++) to Java code.

- **AQS (The "Spring" of Concurrency):** Most Java locks (ReentrantLock, Semaphore, CountDownLatch) are built on AQS. It manages an internal integer `state` and a FIFO queue.
- **LockSupport:** The bridge to the OS. It uses `park()` and `unpark()`, which are significantly more predictable than the internal JVM `ObjectMonitor` logic.
- **Fairness:** Unlike `synchronized`, `ReentrantLock` allows you to set `fair = true`, ensuring the longest-waiting thread gets the lock next, preventing **Starvation**.

3. Atomics & Lock-Free Programming

Atomics skip the JVM and the OS, talking directly to the CPU via **CAS (Compare-And-Swap)** instructions.

- **The CAS Loop:**

```
// Mental model of how atomics work under the hood
public void increment() {
    int expectedValue;
    int newValue;
    do {
        expectedValue = value; // "Check out" current state
        newValue = expectedValue + 1;
    } while (!compareAndSet(expectedValue, newValue)); // "Rebase" if someone else char
}
```

- **ABA Problem:** A major pitfall where a value changes from A to B and back to A. A thread might think "nothing changed" and proceed incorrectly.
- **Solution:** Use `AtomicStampedReference` (adds a version/stamp).

---

4. Code Examples & Explanations

A. Implementing a Custom Non-Reentrant Lock with AQS

This example demonstrates how to use the AQS framework to build a simple mutex.

```
import java.util.concurrent.locks.AbstractQueuedSynchronizer;

public class SimpleMutex {
    // Inner helper class to handle the state
    private static class Sync extends AbstractQueuedSynchronizer {
        // Try to acquire the lock (0 -> 1)
        protected boolean tryAcquire(int arg) {
            if (compareAndSetState(0, 1)) {
                setExclusiveOwnerThread(Thread.currentThread());
                return true;
            }
            return false;
        }

        // Try to release the lock (1 -> 0)
        protected boolean tryRelease(int arg) {
            if (getState() == 0) throw new IllegalMonitorStateException();
            setExclusiveOwnerThread(null);
            setState(0);
            return true;
        }
    }

    private final Sync sync = new Sync();

    public void lock() { sync.acquire(1); }
    public void unlock() { sync.release(1); }
}
```

Explanation: 1. **State (0/1):** We define `0` as unlocked and `1` as locked.
2. **CAS:** `compareAndSetState` ensures that the transition from 0 to 1 is atomic.
3. **Ownership:** We track the thread that owns the lock to prevent others from unlocking it.

B. Multiple Conditions (Bounded Buffer)

This replaces the old `wait/notify` with more granular “Conditions.”

```
import java.util.concurrent.locks.*;

public class BoundedBuffer {
   final Lock lock = new ReentrantLock();
   final Condition notFull  = lock.newCondition();
   final Condition notEmpty = lock.newCondition();

   final Object[] items = new Object[100];
   int putptr, takeptr, count;

   public void put(Object x) throws InterruptedException {
      lock.lock();
      try {
         while (count == items.length)
            notFull.await(); // Wait specifically for "not full"
         items[putptr] = x;
         if (++putptr == items.length) putptr = 0;
         ++count;
         notEmpty.signal(); // Signal that someone can now take
      } finally {
         lock.unlock();
      }
   }
}
```

Explanation: 1. **Wait Loop:** Always use `while` for conditions to protect against **Spurious Wakeups** (where the OS wakes a thread without a signal).
2. **Targeted Signaling:** Instead of waking *everyone* (notifyAll), we only wake the threads waiting to `take` when we `put` something.

---

5. Hands-on Exercises

Exercise 1: The "Stranding" Investigation

Goal: Observe how `synchronized` performs under high contention compared to `ReentrantLock`.

- **Task:** Create 50 threads that increment a shared counter 1,000,000 times using `synchronized`. Repeat with `ReentrantLock`. Measure the time and use a profiler

(like async-profiler) to look for `ObjectMonitor::enter` in the `synchronized` version.
- **Covers:** Understanding the overhead of JVM-managed fat locks vs. OS-managed parking.

Exercise 2: Building a Lock-Free Stack

Goal: Implement a stack using `AtomicReference` without any `synchronized` or `Lock` keywords.

- **Task:** Create a `Node` class. Use `AtomicReference<Node> head`. Implement `push()` and `pop()` using a `do-while` loop with `compareAndSet`.
- **Covers:** Mastery of CAS (Compare-And-Swap) and the "rebase" mental model of lock-free programming.

Exercise 3: ABA Trap & Stamp

Goal: Visualizing the ABA problem.

- **Task:** Simulate a scenario where Thread A reads Value 1. Thread B changes it to 2, then back to 1. Thread A then performs a CAS. Observe it succeeding. Now, refactor the stack to use `AtomicStampedReference` and observe the CAS failing because the stamp (version) has changed.
- **Covers:** Advanced Atomic usage and data integrity in non-blocking systems.

Concurrency Mechanism Comparison Table

Feature	synchronized	ReentrantLock	Atomics (CAS)
Layer of Control	JVM Internal (C++)	Java Code (AQS)	Hardware (CPU)
Performance	High Latency (Contended)	Predictable/Moderate	Lowest Latency
Interaction	Talks to JVM	Talks to OS (via LockSupport)	Talks to CPU Directly
Visibility	Managed by JVM	Java Memory Model	Native CPU Fences
Fairness Support	No (Unfair by default)	Yes (Configurable)	N/A (Lock-free)

Best Use Case	Simple, low-contention code	Complex logic, fairness, or multiple conditions	High-frequency single variable updates

---

Implementation Summary

- **Synchronized:** Uses the `ObjectMonitor` structure in C++. Watch out for "stranding" where threads remain parked even when the lock is available.
- **ReentrantLock:** Built on **AQS (AbstractQueuedSynchronizer)**. It is generally preferred for modern, high-throughput applications because it bypasses the "black box" of JVM monitor inflation.
- **Atomics:** Uses **CAS (Compare-And-Swap)** instructions. It is "Lock-Free" but not necessarily "Wait-Free," as threads may spin/retry in a loop under high contention.

Reference(s)

http://www.youtube.com/watch?v=z4pv8r_uMJM